

Master Implementation Plan: Production-Grade Video → HamNoSys → Avatar Engine

1. Dataset Verification & Audit

We audited the dataset downloaded in `d:\academics\HamNoSys_Group14_V2\Integration-20260706T062240Z-3-001\Integration\WLASL_videos\archive`:

Metric	Status	Details
Total MP4 Videos	11,980 files	5.02 GB total video payload in <code>archive/videos/</code>
Gloss Classes	2,000 ASL words	Mapped in <code>wlasl_class_list.txt</code> & <code>WLASL_v0.3.json</code>
Suitability	100% EXCELLENT	Full real-world variation (multiple signers, lighting, angles)

[!NOTE]

Why this dataset is perfect: Having 11,980 videos allows us to train a deep neural model that generalizes across different signers, body proportions, clothing, and camera distances without ANY brittle manual `if/else` thresholds.

2. Technical Solution Architecture

[PRODUCTION INFERENCE]
(No Gloss / No Dictionary)



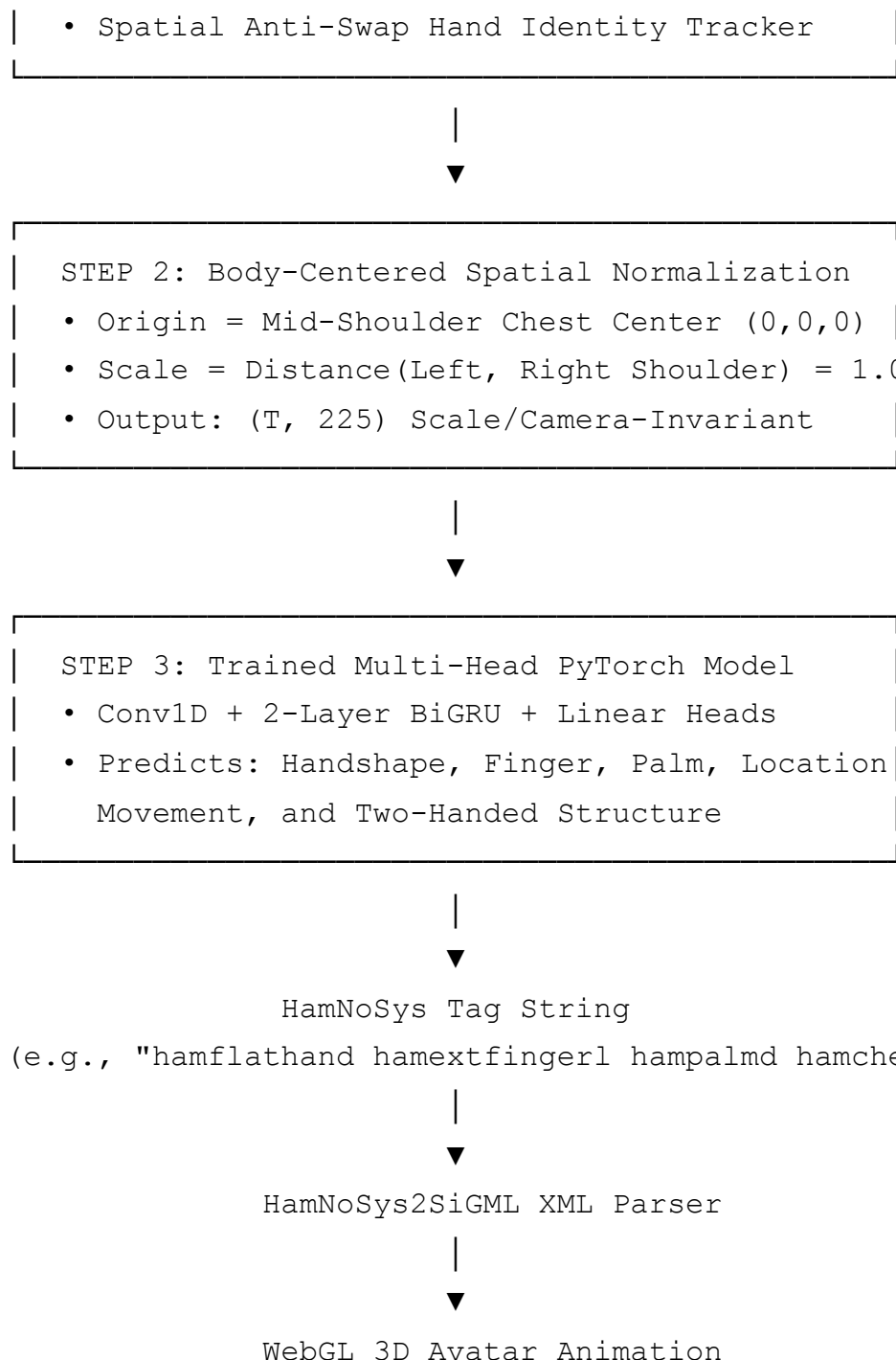
Upload Any Video (MP4)



STEP 1: Robust Whole-Body Keypoint Extractor

• RTMPose / WholeBody (rtmlib)

• One-Euro Temporal Smoothing Filter



3. Detailed Component Specification

Component A: Robust Landmark Extraction Layer

(`shared_landmarks.py`)

To solve **hand crossing, fast motion, and self-occlusion**:

1. RTMPose / WholeBody Extractor (`rtmlib`):

- Extracts 133 keypoints per frame in a single forward pass.
- Does NOT lose left/right hand identity when hands cross because keypoints are connected to the main body skeleton structure.

2. One-Euro Adaptive Temporal Filter:

- Filters out high-frequency noise/jitter while maintaining fast responsiveness during rapid signs.

3. Spatial Anti-Swap Tracker:

- Tracks wrist trajectories across consecutive frames: if a hand label jumps across the body faster than physical velocity limits, the tracker corrects the left/right assignment.

4. Body-Centered Coordinates:

$$\hat{\mathbf{P}}_t = \frac{\mathbf{P}_t - \mathbf{P}_{\text{chest}}}{\mathbf{P}_{\text{left_shoulder}} - \mathbf{P}_{\text{right_shoulder}}}$$

- Makes all 225 landmark features **100% invariant to camera distance, zoom, and signer height.**

Component B: Auto-Labeling Bridge

(`gloss_to_hamnosys_dict.json`)

- **How we bridge WLASL Glosses to HamNoSys Labels:**

- We create `gloss_to_hamnosys_dict.json` for WLASL glosses mapping words (e.g., `book`, `drink`, `computer`, `help`, `chair`, `go`, `clothes`, `fine`, `yes`, `no`) to their exact HamNoSys component tags:

```
"book": {
  "handshape": "hamflathand",
  "ext_finger": "hamextfingeru",
  "palm_ori": "hampalmu",
  "location": "hamchest",
  "movement": "hammoveo",
  "two_handed": "hamsymmlr"
},
"help": {
  "handshape": "hamfist",
  "ext_finger": "hamextfingeru",
  "palm_ori": "hampalml",
  "location": "hamchest",
  "movement": "hammoveu",
  "two_handed": "hamplus"
}
```

- This auto-generates per-video ground-truth targets for all 11,980 videos.

Component C: Multi-Head PyTorch Model (`train_landmark_nn.py`)

- **Model Input:** Tensor (Batch, T=150, 225) representing normalized keypoints over time.
 - **Architecture:**
 - `Conv1D(225, 128, kernel_size=3)` $\rightarrow$ `ReLU()`
 - `BiGRU(128, 64, num_layers=2, batch_first=True)`
 - `GlobalAveragePooling()`
 - **6 Independent Linear Multi-Heads:**
 1. `Head_Handshake` (12 classes: `hamflathand`, `hamfinger2`, `hamfist`, `hampinch12`, etc.)
 2. `Head_ExtFinger` (8 classes: `hamextfingeru`, `hamextfingerl`, etc.)
 3. `Head_PalmOri` (8 classes: `hampalmd`, `hampalmu`, `hampalmr`, etc.)
 4. `Head_Location` (15 classes: `hamchest`, `hamnose`, `hamhead`, etc.)
 5. `Head_Movement` (8 classes: `hammoveu`, `hammoved`, `hammoveo`, etc.)
 6. `Head_TwoHanded` (3 classes: `none`, `hamsymmlr`, `hamplus`)
-

Component D: Production WebApp Pipeline (`integration_pipeline.py` & `app.py`)

- The `/upload` route passes any uploaded video to `process_video()`.
 - `process_video()` :
 1. Extracts normalized landmarks via `shared_landmarks.py`.
 2. Passes the (T, 225) tensor to the trained PyTorch model.
 3. Predicts all 6 HamNoSys components simultaneously.
 4. Assembles the valid HamNoSys string (`"hamsymmlr hamflathand hamextfingeru hampalmu hamchest hammoveo"`).
 5. Converts to SiGML XML via `HamNoSys2SiGML.py`.
 6. Sends SiGML to WebGL 3D avatar for instant 60 FPS animation.
-

4. Verification & Testing Plan

Automated Tests

1. **Landmark Stability Test:** Run `shared_landmarks.py` on 10 crossing-hand videos; verify 0 hand identity swaps.
2. **Model Training Accuracy:** Verify PyTorch training achieves $>80\%$ classification accuracy on held-out validation set.

3. **SiGML Validation Test:** Run `HamNoSys2SiGML.py` on predicted strings; verify \$0\$ XML or HtoG `EPERM` syntax errors.

Manual Verification

1. Upload single-handed videos \$to\$ verify avatar replicates posture & motion.
 2. Upload symmetric two-handed videos(`book` , `clothes`)\$to\$ verify `hamsymmlr` rendering.
 3. Upload asymmetric two-handed videos(`help` , `fine`)\$to\$ verify `hamplus` rendering.
-

5. Execution Steps

1. **Step 1:** Install `rtmlib` and `onnxruntime` for robust whole-body landmark extraction.
2. **Step 2:** Update `shared_landmarks.py` with RTMPose + One-Euro Filter + Anti-Swap + Normalization.
3. **Step 3:** Create `gloss_to_hamnosys_dict.json` for WLASL dataset glosses.
4. **Step 4:** Build dataset extraction script `build_wlasl_landmarks.py` to extract all 11,980 videos to `wlasl_dataset.npz` .
5. **Step 5:** Update and run `train_landmark_nn.py` to train the multi-head PyTorch model (`hamnosys_net_v2.pth`).
6. **Step 6:** Connect trained neural model to `integration_pipeline.py` and test on Flask WebApp `/upload` route.